SAVEETHA SCHOOL OF ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT | CO1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Department	B.E. Computer Science and Engineering (Cyber Security)
Course	CSA0421 – Operating Systems
Assignment	Assignment 1 (Tasks 1)

TASK 1 – EVALUATE THE REQUIREMENTS OF EACH COMPUTING ENVIRONMENT

The first design activity is to identify what each environment must achieve and rank those needs. The ranking is based on the consequences of failure, not simply on convenience or hardware cost.

1.1 Ranked requirements

Environment	Ranked requirements (highest first)	Justification
Bedside patient monitors	1. Hard deadline / deterministic response; 2. Patient safety and fault recovery; 3. Reliability; 4. Basic security; 5. Low memory and power	A delayed alarm can cause direct patient harm. The response must be bounded and local. Unnecessary background tasks must not interfere.
Nurses' handheld tablets	1. Battery life; 2. Secure and reliable access; 3. Usability / mobility; 4. Responsive I/O; 5. Manageable updates	A tablet is used throughout a shift and moves between wards. It needs power management, encryption, authentication and fast access to patient records.
Doctors' desktop workstations	1. Responsiveness; 2. Security / privacy; 3. Application compatibility; 4. Reliability; 5. Manageability	Doctors use imaging viewers, electronic records and clinical applications. Desktop multitasking and user-interface quality matter, but record confidentiality remains essential.
Central patient database	1. Availability and data integrity; 2. Security and auditability; 3. Transaction throughput; 4. Recoverability; 5. Maintainability	A database outage affects clinical work across the hospital. Transactions must remain correct, and fail-over, backups and controlled patching are required.

Environment	Ranked requirements (highest first)	Justification
MRI imaging server	1. Throughput / parallel computation; 2. Memory and storage bandwidth; 3. Reliability; 4. Availability; 5. Security	MRI reconstruction processes large datasets. Multiple processors, sufficient RAM, fast storage and DMA-capable I/O increase throughput.
Partner-hospital link	1. Confidentiality and integrity; 2. Availability; 3. Interoperability; 4. Throughput; 5. Controlled latency	The link is a distributed-system connection. Secure, authenticated exchange and retry/recovery are more important than a 10 ms deadline.

1.2 Conflicting requirements

- Fast deterministic response versus high utilization: general-purpose schedulers improve average throughput by sharing CPU time, but a monitor needs a predictable maximum delay. Background logging, display refresh or network tasks must never block alarm handling.
- High availability versus maintenance: a database must be patched and backed up without a prolonged service interruption. Redundancy makes maintenance more complex because replication state and fail-over must be coordinated.
- Battery life versus responsiveness: a tablet saves power by dimming the screen, reducing CPU frequency and suspending idle tasks. These measures are useful, but aggressive sleep must not delay a required clinical notification.
- Parallel throughput versus shared-resource contention: additional imaging processors improve performance only until memory bandwidth, storage I/O or serial portions of the application become bottlenecks.
- Security versus convenience: stronger authentication, encryption and audit logging add processing and user steps. They should be designed efficiently, not removed from clinical systems.

1.3 Patient-safety-critical requirements

The bedside alarm is the most time-critical function because the assignment specifies a 10 ms response. The monitor should use a hard-real-time scheduling policy, bounded interrupt handling, a small trusted software base, watchdog supervision and a local alarm path. The database is also safety significant because unavailable or incorrect records can affect clinical decisions, but it is primarily an availability and integrity problem rather than a 10 ms hard-deadline problem. Imaging delays can disrupt clinical workflow, yet they normally do not justify the same hard-real-time design as the bedside alarm.

1.4 Lessons from operating-system history

Historical development	**Lesson**	**Hospital application**
Batch systems	Jobs can be collected and processed efficiently, but interactive response is not guaranteed.	Use batch-style queues for backups, archive jobs and non-urgent image processing, not for bedside alarms.
Time-sharing systems	CPU time is divided among many users to give interactive response.	Use multi-user time-sharing on the database, desktop and shared application servers, with quotas and priorities.
Personal / desktop systems	Graphical interfaces and device support improve individual productivity.	Use a desktop OS on doctors' workstations for clinical applications and imaging viewers.
Mobile / handheld systems	Power management, wireless connectivity and touch interaction are central.	Use a managed handheld OS on nurses' tablets with encryption and controlled background activity.
Real-time / embedded systems	Correctness depends on both the result and the time at which it is produced.	Use a real-time embedded OS on patient monitors to enforce the 10 ms deadline.

1.5 Alternative: one general-purpose OS everywhere

If all devices used one general-purpose OS, administration might initially appear simpler because staff would learn one platform and IT could use common deployment tools. However, the design would impose desktop/server features on small monitors and mobile devices, and it would not automatically provide a proven worst-case alarm deadline. Therefore, reject one-size-fits-all; standardize management policies rather than the kernel image.

Evaluation point	Single general-purpose OS	Specialized OS plan
Worst-case alarm delay	A normal general-purpose scheduler may have variable and unbounded application delay from the application's point of view. A tuned real-time kernel can reduce jitter, but an ordinary configuration does not prove a hard 10 ms bound.	A configured hard-RTOS, bounded drivers, priority analysis and local alarm path can be engineered and tested against a 10 ms deadline.
Memory footprint	A full general-purpose stack commonly needs substantially more RAM and storage than a small embedded kernel; exact size depends on selected distribution and services.	A monitor image can omit GUI, browser, general-purpose services and large filesystems, reducing memory and attack surface.
Battery / mobility	Desktop-oriented background services and drivers can increase idle power and update load.	Handheld OS power management can coordinate radio, screen, CPU and application lifecycle.